Estratégia Concursos

Aula 43

Polícia Federal (Agente de Polícia) Passo Estratégico de Informática - Prof Diego Carvalho

Autor:
Diego Carvalho

09 de Abril de 2025

98276239072 - FABIANO CARBONELL DA SILVA

Índice

1) O que é mais cobrado no assunto - Python - CEBRASPE 3

2) Roteiro de Revisão - Python 4

3) Aposta Estratégica - Python 12

4) Questões Estratégicas - Python - CEBRASPE 13

5) Questionário de Revisão - Python 21

6) Lista de Questões Estratégicas - Python - CEBRASPE 31

7) Gabarito de Questões Estratégicas - Python - CEBRASPE 34

8) Referências Bibliográficas - Python 35

98276239072 - FABIANO CARBONELL DA SILVA

O QUE É MAIS COBRADO DENTRO DO ASSUNTO?

Considerando os tópicos que compõem o nosso assunto, possuímos a seguinte distribuição percentual:

TÓPICO	% DE COBRANÇA [CEBRASPE]
Conceitos Básicos	04%
Representações de Algoritmos	02%
Noções de Python	08%
Tipos de Operadores	04%
Funções	09%
Comentários	02%
Tipos e Variáveis	38%
Controle de Fluxo	10%
Orientação a Objetivos	13%
Arquivos	10%

98276239072 - FABIANO CARBONELL DA SILVA

Roteiro de Revisão e Pontos do Assunto que Merecem Destaque

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

CONCEITOS BÁSICOS	DESCRIÇÃO
INSTRUÇÃO	É um conjunto de regras ou normas simples e bem definidas para a realização ou emprego de algo, indicando ao computador uma ação ou função elementar a ser executada.
ALGORITMO	Um conjunto predeterminado e bem definido de passos destinados à solução de um problema, com um número finito de etapas.
REPRESENTAÇÃO	Algoritmos podem ser representados por: linguagem natural, linguagem de máquina, linguagem de programação, pseudocódigo e fluxograma.

Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, conhecida por sua sintaxe clara e simples, que facilita a legibilidade do código. Ela suporta múltiplos paradigmas, como programação orientada a objetos, funcional e procedural. Python é amplamente utilizado em desenvolvimento web, ciência de dados, automação, inteligência artificial e muitas outras áreas, devido à sua vasta biblioteca padrão e grande comunidade de suporte.

PRINCIPAIS CARACTERÍSTICAS	DESCRIÇÃO
ALTO NÍVEL	Trata-se de uma linguagem de alto nível porque é uma linguagem com nível de abstração bastante elevado, bem longe do código de máquina e mais próximo da linguagem humana. Em outras palavras, o Python está muito mais próximo à forma humana de pensar/escrever do que dos comandos entendidos por um processador.
INTERPRETADA	Trata-se de uma linguagem interpretada porque seu código-fonte é executado linha a linha por um interpretador (software que traduz uma linguagem em outra) e depois pelo sistema operacional, isto é, seu código-fonte não precisa ser totalmente transformado em código de máquina para somente depois ser executado.
MULTIPARADIGMA	Trata-se de uma linguagem multiparadigma porque suporta mais de um paradigma de programação. No caso, eles são: imperativo, procedural, funcional e orientado a objetos. Paradigmas são formas diferentes de resolver problemas – alguns utilizam funções, outros utilizam objetos, outros procedimentos e assim por diante.
CASE SENSITIVE	Trata-se de uma linguagem case-sensitive, isto é, capaz de diferenciar maiúsculas e minúsculas. Em outras palavras, isso significa que as variáveis **nome**, **Nome** e **NOME** são diferentes entre si, uma vez que possuem diferenças de caixa em sua grafia.

98276239072 - FABIANO CARBONELL DA SILVA

OUTRAS CARACTERÍSTICAS DO PYTHON
Python permite utilizar ponto-e-vírgula (;) para delimitar comandos, mas de forma opcional – assim como em outras linguagens como JavaScript e Typescript.
Python é distribuída sob uma licença própria (compatível com a GPL), que permite a distribuição – comercial ou não – tanto da linguagem quanto de aplicações desenvolvidas nela, em formato binário ou código fonte, bastando cumprir a exigência de manter o aviso de copyright.
Python permite que programas sejam compilados para um formato portável chamado de bytecode. Essa característica faz com que programas escritos nessa linguagem com uma biblioteca padrão sejam executadas da mesma forma em diversos sistemas operacionais que possuam um software interpretador de Python.
Python possui uma interface com muitas bibliotecas e sistemas de janela, sendo extensível em C/C++. Além disso, pode ser utilizada como linguagem de extensão para aplicações que necessitam de uma interface programável (muito comum em aplicativos, jogos, processamento de textos, dados científicos e motores de busca).
Python possui funcionalidades para expressões regulares; sockets; threads; data/tempo; analisadores XML; analisadores de arquivos de configuração; manipulação de arquivos e diretórios; persistência de dados; unidades de testes; bibliotecas clientes para os protocolos HTTP, FTP, IMAP, SMTP e NNTP.
Python suporta o paradigma orientado a objetos com todos os seus componentes, tais como herança (simples ou múltipla), polimorfismo, sobrescrita, encapsulamento, abstração, reflexão, introspecção, etc! Não é necessário entender o que isso significa exatamente, apenas saber que ele suporta e que tudo em Python é um objeto.

A identação em Python é fundamental, pois define blocos de código, como loops, funções e condicionais. Diferente de outras linguagens que utilizam chaves ou palavras-chave, Python exige que os blocos sejam organizados por recuos (geralmente 4 espaços ou um tab). Um erro na identação resultará em falha de execução. Essa abordagem promove legibilidade e organização, sendo uma característica essencial da linguagem.

```
1: i = 0
2: while i < 3:
3:     print(i)
4:     if i % 2 == 0:
5:         print("é par")
6:     i = i + 1
7:     print("Acabou o loop")
```

```
1: i = 0
2: while i < 3:
3:     print(i)
4:     if i % 2 == 0:
5:         print("é par")
6:     i = i + 1
7:     print("Acabou o loop")
```

Assim como na matemática, operadores são elementos utilizados em linguagens de programação para realizar operações. Existem quase 40 operadores em Python, mas nós não vamos ver todos porque diversos deles possuem um péssimo custo/benefício e não valerá a pena! Vamos nos ater – portanto – aos operadores mais cobrados em prova. A seguir serão apresentadas categorias de operadores com suas descrições e exemplos.

SÍMBOLO	OPERADORES ARITMÉTICOS	EXEMPLO

98276239072 - FABIANO CARBONELL DA SILVA

+	Soma	10 **+** 10 = **20**.
-	Subtração	20 **-** 20 = **0**.
*****	Multiplicação	15 ***** 25 = **375**.
/	Divisão	25 **/** 4 = **6,25**.
//	Divisão com Piso	25 **//** 4 = **6** (6,25 arredondado para baixo).
******	Exponenciação	2 ****** 5 = **32**.
%	Módulo/Resto	30 **%** 7 = **2** (resto da divisão).

SÍMBOLO	OPERADORES DE ATRIBUIÇÃO	EXEMPLO
=	Atribuição Simples	**x = 10** é o mesmo que atribuir o valor **10** a **x**.
+=	Atribuição com Soma	**x += 1** é o mesmo que **x = x + 1**, logo **x = 10 + 1 = 11**.
-=	Atribuição com Subtração	**x -= 6** é o mesmo que **x = x - 6**, logo **x = 11 - 6 = 5**.
***=**	Atribuição com Produto	**x *= 2** é o mesmo que **x = x*2**, logo **x = 5 * 2 = 10**.
/=	Atribuição com Divisão	**x /= 5** é o mesmo que **x = x / 5**, logo **x = 10 / 5 = 2**.

SÍMBOLO	OPERADORES DE COMPARAÇÃO	EXEMPLO
==	Igual	**x == y** retorna **Falso**.
!=	Diferente	**x != y** retorna **Verdadeiro**.
>	Maior que	**x > y** retorna **Falso**.
<	Menor que	**x < y** retorna **Verdadeiro**.
>=	Maior ou igual a	**x >= y** retorna **Falso**.
<=	Menor ou igual a	**x <= y** retorna **Verdadeiro**.

SÍMBOLO	OPERADORES LÓGICOS	EXEMPLO
and	Retorna Verdadeiro se ambas condições forem Verdadeiras	x **==** 10 **and** y **>** 20 retorna **Falso**.
or	Retorna Verdadeiro se uma das condições for Verdadeira	x **==** 10 **or** y **>** 20 retorna **Verdadeiro**.
not	Inverte o resultado de uma condição qualquer	**not**(x **==** 10 **or** y **>** 20) retorna **Falso**.

SÍMBOLO	OPERADORES DE ASSOCIAÇÃO	EXEMPLO
in	Retorna Verdadeiro se o conjunto de valores estiver presente no objeto	**x = [“**Flamengo**”, “**Vasco**”]** **y = “**Botafogo**” in x** **print(y)** retornará **Falso** porque **Botafogo** não está na lista.
not in	Retorna Verdadeiro se o conjunto de valores não estiver presente no objeto	**x = [“**Flamengo**”, “**Vasco**”]** **y = “**Flamengo**” not in** **print(y)** retornará **Falso** porque **Flamengo** está na lista.

98276239072 - FABIANO CARBONELL DA SILVA

Uma função é um bloco de código que é executado apenas quando é invocado. São três passos bem simples: (1) uma função é invocada; (2) é passado algum dado como parâmetro ou não; (3) e algum dado é retornado como resultado. No Python, uma função é definida utilizando a palavra-chave def. Sempre que você vir essa palavra em um código-fonte, saiba que ela está definindo uma função (lembrando que a utilização de chaves para delimitar o bloco é facultativa).

```
Sintaxe: def NomeFunção(argumentos):
```

```
Sintaxe: lambda lista_de_argumentos : expressão
```

A utilização de comentários é uma prática comum em programação. Seu objetivo é adicionar descrições em partes especificas do código, seja para documentá-lo, seja para descrevê-lo, ou mesmo, para marcar que uma determinada linha, ou um conjunto de linhas, não devem ser executadas. Podem ser utilizados para documentar programas, classes, funções, variáveis, constantes ou o que mais você desejarmos.

```
# Comentário para documentar a função "Olá, Mundo!"
print("Olá, Mundo!")
```

```
'''
Comentário para documentar
a função "Olá, Mundo!"
com mais de uma linha
e aspas simples
'''

print("Olá, Mundo!")
```

```
"""
Comentário para documentar
a função "Olá, Mundo!"
com mais de uma linha
e aspas duplas
"""

print("Olá, Mundo!")
```

REGRAS DE NOMENCLATURA DE VARIÁVEIS
(1) Começar necessariamente com uma letra ou _ (underline); (2) Jamais devem começar com um número; (3) Conter caracteres alfanuméricos e undelines (a-z, A-Z, 0-9 e _); (4) São Case-Sensitive, isto é, capazes de diferenciar maiúscula de minúscula.

NOMES VÁLIDOS DE VARIÁVEIS	NOMES INVÁLIDOS DE VARIÁVEIS
nomevariavel = "Diego"	2nomevariavel = "Diego"
nome_variavel = "Diego"	nome-variavel = "Diego"
_nome_variavel = "Diego"	nome variavel = "Diego"
nomeVariavel = "Diego"	nome#variavel = "Diego"

Vejamos agora os principais tipos de dados suportados:

98276239072 - FABIANO CARBONELL DA SILVA

CATEGORIA	TIPO	PYTHON	EXEMPLO
BOOLEANO	Booleano	Bool	`x = True` `x = False`
NUMÉRICO	Inteiro	Int	`x = 10` `x = -5`
	Ponto Flutuante	Float	`x = 10.7` `x = -2.8`
	Complexo	Complex	`x = 345j` `x = 2-9j`
TEXTUAL	Texto	Str	`x = 'texto'` `x = "texto"`
COLEÇÃO/ SEQUÊNCIA	Lista	List	`x = [4, 8]` `x = list()`
	Tupla	Tuple	`x = (5, 10)` `x = tuple()`
	Set	Set	`x = {2, 4}` `x = set( )`
	Dicionário	Dictionary	`x = {'nome': 'Diego', idade: 31}`

Para armazenar múltiplos itens dentro de uma única estrutura, utilizamos estruturas de dados. No caso do Python, essas estruturas de dados são chamadas de coleções (collections). Vejamos:

TIPO DE COLEÇÃO	TRADUÇÃO	DESCRIÇÃO
LIST	LISTA	Trata-se de uma coleção de valores ordenados, mutáveis e indexáveis que pode conter valores duplicados.
TUPLE	TUPLA	Trata-se de uma coleção de valores ordenados, imutáveis e indexáveis que pode conter valores duplicados (também podem ser chamadas de sequências).
SET	CONJUNTO	Trata-se de uma coleção de valores desordenados, mutáveis e não indexáveis que não pode conter valores duplicados.
DICTIONARY	DICIONÁRIO	Trata-se de uma coleção de valores ordenados, mutáveis e indexáveis que não pode conter valores duplicados.

Vejamos uma tabela comparativa de cada coleção:

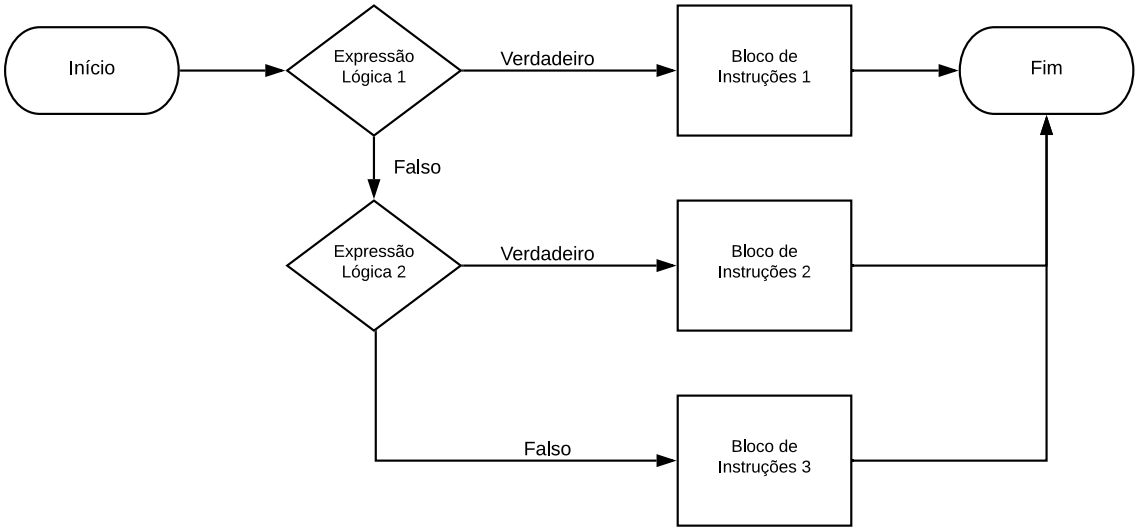
CRITÉRIO	LIST	TUPLE	SET	DICTIONARY
ORDENAÇÃO	Ordenada	Ordenada	Não ordenada	Ordenada
MODIFICAÇÃO	Mutável	Imutável	Mutável	Mutável
DUPLICATAS	Permite duplicatas	Permite duplicatas	Não permite duplicatas	Não permite duplicatas
INDEXAÇÃO	Por inteiro	Por inteiro	Não indexada	Por string

98276239072 - FABIANO CARBONELL DA SILVA

DELIMITADOR	Entre colchetes [ ]	Entre parênteses ( )	Entre chaves { }	Entre chaves { }

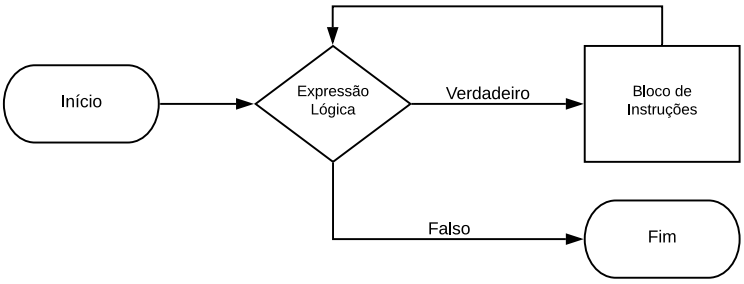
Vamos estudar agora controles de fluxo. Comecemos por estruturas de decisão:

```
if expressão-lógica1:
    bloco-de-instruções1
elif expressão-lógica2:
    bloco-de-instruções2
else:
    bloco-de-instruções3
```



Agora vamos estudar estruturas de repetição. Comecemos pelo *while*:

```
while expressão-lógica:
    bloco-de-instruções
```



Agora vamos ver a estrutura do *for*:

```
for variável-de-controle in variavel-colecao
    bloco-de-instruções
```


98276239072 - FABIANO CARBONELL DA SILVA

ALTERAÇÕES DE FLUXO	DESCRIÇÃO
BREAK	Interrompe o loop completamente e vai para a linha seguinte ao término do loop.
CONTINUE	Interrompe a iteração atual do loop e volta para o teste de condição do loop.

Em Python, a manipulação de arquivos é feita usando as funções integradas como open(), que permite abrir arquivos para leitura ('r'), escrita ('w') ou adição de conteúdo ('a'). Após a manipulação, é importante fechar o arquivo com close() ou usar o gerenciamento automático com with, que garante o fechamento. Python suporta leitura e escrita de arquivos de texto e binários, oferecendo funções como read(), write(), e readlines() para lidar com o conteúdo.

```
Sintaxe: open(NomeDoArquivo, Modo)  #Permite abrir/criar um arquivo
```

MODO DE USO	SIGNIFICADO	DESCRIÇÃO
'r'	READ	Esse parâmetro permite abrir um arquivo para leitura a partir de sua posição inicial sem que o arquivo seja truncado e retorna um erro caso o arquivo não exista (é o modo *default*).
'w'	WRITE	Esse parâmetro permite abrir um arquivo para escrita a partir de sua posição inicial após o arquivo ser truncado e, caso o arquivo não exista, ele é criado.
'a'	APPEND	Esse parâmetro permite abrir um arquivo para escrever/adicionar dados a partir de sua posição final sem que o arquivo seja truncado e, caso o arquivo não exista, ele é criado.
'x'	CREATE	Esse parâmetro permite criar um arquivo especificado e, caso o arquivo exista, ele retorna um erro.

MODO DE USO	SIGNIFICADO	DESCRIÇÃO
'r+'	READ/WRITE	Esse parâmetro permite abrir um arquivo para leitura e escrita de dados a partir de sua posição inicial sem que o arquivo seja truncado e retorna um erro caso o arquivo não exista.
'a+'	APPEND/READ	Esse parâmetro permite abrir um arquivo para escrever/adicionar e ler dados a partir de sua posição final sem que o arquivo seja truncado e, caso o arquivo não exista, ele é criado.
'w+'	WRITE/READ	Esse parâmetro permite abrir um arquivo para escrita e leitura de dados a partir de sua posição inicial após o arquivo ser truncado e, caso o arquivo não exista, ele é criado.

RESUMO	'x'	'r'	'r+'	'w'	'w+'	'a'	'a+'
CRIAR	X			X	X	X	X
LER		X	X		X		X
ESCREVER			X	X	X	X	X
TRUNCAR				X	X		

98276239072 - FABIANO CARBONELL DA SILVA

A PARTIR DO INÍCIO		X	X	X	X		
A PARTIR DO FIM						X	X

MODO DE USO	SIGNIFICADO	DESCRIÇÃO
't'	TEXT	Permite manipular o arquivo de texto simples – aqueles que contenham apenas caracteres alfanuméricos sem formatação (é o *default*).
'b'	BINARY	Permite manipular o arquivo como um binário – aqueles restantes que tratam de imagens, vídeos, áudios, documentos, planilhas, etc.

MÉTODO	DESCRIÇÃO
`read()`	Retorna o conteúdo do arquivo como uma única string.
`readline()`	Retorna uma linha do texto a cada chamada na ordem em que aparecem no arquivo.
`readlines()`	Retorna uma lista de valores de string do arquivo, sendo que cada string corresponde a uma linha do texto.

98276239072 - FABIANO CARBONELL DA SILVA

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais.

Eu listei abaixo os pontos com maior probabilidade de cobrança no contexto de **Linguagem Python**. Estas são as minhas apostas:

1. **Variáveis** e **constantes** são temas prováveis, com foco na diferença entre elas e no conceito de que variáveis podem ter seus valores alterados durante a execução de um programa.

2. Pode cair uma questão sobre estruturas de controle de fluxo, como **if**, **else**, **while** e **for**, que permitem ao programa tomar decisões com base em condições ou repetir blocos de código.

3. Aposto em questões sobre operadores aritméticos e lógicos, como **+**, **-**, *****, **/**, e operadores de comparação como **==**, **!=**, **>**, **<**, que são fundamentais em expressões de controle.

4. Vale revisar o **conceito de função** em programação, que é um bloco de código executado quando chamado, podendo receber parâmetros e retornar valores.

5. Aposto que haverá perguntas sobre o conceito de **escopo de variáveis**, diferenciando o escopo local e global e sua importância na execução de funções.

6. Exceções em programação também são um ponto relevante, destacando o uso da estrutura **try-except** para tratar erros durante a execução do código.

7. Aposto em questões sobre **funções lambda**, que são funções anônimas de uma linha, úteis para simplificar códigos, especialmente em expressões temporárias.

8. Funções em Python e sua definição com a palavra-chave **def** devem aparecer, destacando a importância de invocar e passar parâmetros para funções.

98276239072 - FABIANO CARBONELL DA SILVA

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

```
a = “Cagepa”
x = len(a)
x// = 3
print(x)
```

1. (CESPE / CAGEPA-PB – 2024) Assinale a opção que corresponde ao resultado da execução do código precedente, desenvolvido em Python.

a) 0,75
b) 3
c) 2
d) 0
e) Cag

Comentários:

Na primeira linha, a variável a recebe a string “Cagepa”; na segunda linha, a função len(a) retorna o comprimento da string a. No caso da string "Cagepa", a quantidade de caracteres é 6. Então, x vai receber o valor 6; na terceira linha, x//=3 é o mesmo que x = x//3, sendo que // é o operador de divisão inteira (sem parte decimal). Logo, temos que x = 6//3 = 2; na última linha, apenas imprime-se o valor de x (2).

Ocorre que temos um probleminha: o correto deveria ser x //= 3, mas o código é x// = 3. Isso resultaria em um erro de sintaxe, mas vamos aguardar o gabarito definitivo.

Gabarito: Letra C

2. (CESPE / SEPLAG-CE – 2024) Considerando linguagens de programação, julgue o item que se segue. Considere o código Python a seguir.

```
x = ("laranja","lima","taiti")
y = ["laranja","lima","taiti"]
print (type(x))
print (type(y))
```

Esse código, após executado, apresentará o seguinte resultado.

<class 'list'>

98276239072 - FABIANO CARBONELL DA SILVA

<class 'array'>

Comentários:

O código apresentado define duas variáveis, x e y, com coleções de frutas, e em seguida imprime os tipos dessas variáveis. Vamos analisar:

x = ("laranja", "lima", "taiti"): Aqui, x é uma tupla (tuple), pois está delimitada por parênteses.
y = ["laranja", "lima", "taiti"]: Aqui, y é uma lista (list), pois está delimitada por colchetes.

Logo, o resultado correto ao executar esse código será:

<class 'tuple'>
<class 'list'>

Gabarito: Errado

3. (CESPE / ANA – 2024) A linguagem Python é considerada dinamicamente tipada, pois os tipos das variáveis de seus programas podem ser modificados durante a execução.

Comentários:

A linguagem Python é de fato dinamicamente tipada, o que significa que o tipo de uma variável é determinado em tempo de execução, e não precisa ser declarado explicitamente. Além disso, o tipo de uma variável pode ser modificado ao longo do programa, ou seja, uma mesma variável pode conter diferentes tipos de valores em diferentes momentos. Vejamos um exemplo:

```
x = 10 # 'x' é um número inteiro
x = "Python" # 'x' agora é uma string
```

Gabarito: Correto

4. (CESPE / TRT8 – 2022) Na linguagem de programação Python, as funções:

a) retornam somente objetos.
b) não aceitam parâmetros opcionais (com *defaults*).
c) não podem ter suas propriedades alteradas.
d) não aceitam *doc strings*.
e) aceitam que os parâmetros sejam passados com nome, não importando a ordem em que os parâmetros foram passados.

Comentários:

(a) Correto, tudo é objeto em Python; (b) Errado, claro que aceitam parâmetros opcionais – há funções que não obrigam a utilização de parâmetros; (c) Errado, é claro que podem ter suas propriedades alteradas; (d) Errado, aceitam – sim – *doc strings*. Lembrando que *doc strings* são strings inseridas geralmente no início do código-fonte com o intuito de fornecer uma explicação

98276239072 - FABIANO CARBONELL DA SILVA

sobre seu funcionamento; (e) Correto, as funções realmente aceitam que os parâmetros sejam passados com nome, sem importar a ordem.

Ué, Diego! Então há duas alternativas corretas? Na minha visão, sim! O gabarito ainda é preliminar e o examinador considerou correta apenas a última alternativa, mas – para mim – não há erro no primeiro item. Funções retornam literais (que são objetos); retornam outras funções (que também são objetos); enfim... tudo é objeto, logo as funções retornam somente objetos.

Gabarito: Letra E

5. (CESPE / PC-PB - 2022) Python é uma linguagem procedural que utiliza quatro tipos de dados predefinidos para lidar com coleções: conjuntos, dicionários, listas e tuplas. A respeito desses tipos de dados, julgue os itens a seguir.

I O conjunto permite o armazenamento de uma tupla, mas não o de uma lista.
II A tupla é idêntica à lista, exceto pela forma mais simples com que sua declaração é realizada.
III A lista é um tipo de dados variável que permite a alteração de seus elementos após a sua criação.

Assinale a opção correta.

a) Apenas o item I está certo.
b) Todos os itens estão certos.
c) Apenas o item II está certo.
d) Apenas os itens I e III estão certos.
e) Apenas os itens II e III estão certos.

Comentários:

(I) Correto. De fato, um conjunto (set) pode armazenar uma tupla, mas não pode armazenar uma lista, isso ocorre porque os conjuntos são mutáveis, mas não podem armazenar elementos mutáveis entre si; (II) Errado. Outra diferença é que uma tupla é imutável, enquanto uma lista é mutável; (III) Correto, a lista realmente permite a alteração de seus elementos após a sua criação.

Gabarito: Letra D

6. (CESPE / DPE-RO – 2021) Na linguagem Python, são consideradas sequências mutáveis as:

a) strings.
b) cadeias.
c) ranges.
d) tuplas.
e) listas.

Comentários:

98276239072 - FABIANO CARBONELL DA SILVA

(a) Errado, strings não são sequências; (b) Errado, cadeias são strings; (c) Errado, a função *range* é utilizada para retornar uma sequência de números; (d) Errado, tuplas são sequências imutáveis; (e) Correto, listas são sequências mutáveis.

Gabarito: Letra E

7. (CESPE / SERPRO – 2021) As tuplas, embora sejam semelhantes às listas, estão limitadas a, no máximo, cinco níveis.

Comentários:

É possível incorporar uma tupla dentro da outra, permitindo a criação de hierarquias com tuplas. As tuplas são imutáveis, o que significa que não é possível alterá-las. Você pode criar uma tupla com o mesmo nome e modificá-la de algum modo, mas não pode modificar uma já existente. As listas são mutáveis, o que significa que é possível alterá-las. Assim, inicialmente uma tupla pode parecer uma desvantagem, mas a imutabilidade tem várias vantagens, como ser mais segura e rápida. Além disso, objetos imutáveis são mais fáceis de usar com vários processadores.

Ex: Tupla = (1, 2, 3, (4, 5, 6, (7, 8, 9))) é aninhada em três níveis de profundidade. O primeiro nível consiste nos valores 1, 2, 3 e uma tupla. O segundo, nos valores 4, 5, 6 e ainda outra tupla. O terceiro nível consiste nos valores 7, 8 e 9. No entanto, não existe limitação de nenhuma delas em relação a quantidade máxima de níveis suportados.

Gabarito: Errado

8. (CESPE / SERPRO – 2021) Listas são coleções alteráveis de qualquer tipo de objeto – como, por exemplo, outras listas – capazes de gerar um efeito top-down sem limite de níveis.

Comentários:

Listas são coleções mutáveis em que cada elemento pode ser de qualquer tipo, inclusive outras listas, permitindo a criação de listas aninhadas (listas dentro de listas). No entanto, o termo "efeito top-down" é vago e desnecessário aqui, já que o ponto importante é a capacidade de aninhamento.

Gabarito: Correto

9. (CESPE – SEED-PR / 2021) Em Python, quando mais de um operador aparece em uma expressão, a ordem de avaliação depende das regras de precedência de cada linguagem. Assim, ao programar em Python, além de observar essas regras, é preciso considerar, ainda, a forma como a linguagem representa seus operadores, conforme demonstrado nos comandos a seguir.

x=7*32%4 print (x)**

Assinale a opção que corresponde à saída que o compilador Python apresentará para os comandos em questão:

98276239072 - FABIANO CARBONELL DA SILVA

a) 1
b) 3
c) 7
d) 15
e) 15.75

Comentários:

Quando uma expressão possui vários operadores, devemos obedecer a ordem de precedência: Parênteses, Exponenciação, Multiplicação/Módulo/Divisão, Adição/Subtração. Logo, vamos analisar os itens:

x = 7*3**2%4
x = 7*9%4
x = 63%4
x = 3

Gabarito: Letra B

10. (CESPE – SEED-PR / 2021) Considere o programa a seguir, na linguagem Python.

```
if 5 > 2
{
print("True!")
}
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado.

True!

Comentários:

Opa... está faltando os dois-pontos após o critério da função condicional. Deveria ser:

```
if 5 > 2:
    {
    print("True!")
    }
```

Professor, não está errada a utilização de chaves? Não, elas são facultativas (e geralmente não são utilizadas).

Gabarito: Errado

11. (CESPE / Polícia Federal – 2021) O código Python a seguir apresenta como resultado "TRUE".

98276239072 - FABIANO CARBONELL DA SILVA

```
x = bool(-3)
y = bool("True"*x)
z = bool("False")
print(x and y and z)
```

Comentários:

A função bool() retorna sempre **True**, exceto ser o parâmetro de entrada for: **False**, 0 ou vazio. *Como assim, Diego?* Vejam só:

```
bool(False) = False
bool(0) = False
bool() = False
```

Em todos os outros casos, ele retornará **True**! Lembrem-se que **False** é diferente "False": o primeiro é um booleano e o segundo é uma string. Logo...

```
bool(False) = False
bool("False") = True
```

Dito isso, vejamos qual seria o resultado do código:

```
bool(-3) = True          #Não é nenhuma das exceções, logo retornará True
bool("True"*x) = True    #String é True, multiplicado por x (True) retornará True
bool("True") = True              #String sempre retornará True
print(x and y and z)     #x, y, z = True, logo x and y and z = True
```

O gabarito preliminar veio como verdadeiro, porque realmente o resultado é True. No entanto, a banca mudou o entendimento no gabarito definitivo porque a palavra True veio entre aspas – o que seria uma string e, não, um booleano. Eu acho que as aspas deixaram a questão ambígua e, não, errada. No entanto, o gabarito definitivo foi errado...

Gabarito: Errado

12. (CESPE – Polícia Federal / 2018) Considere o programa a seguir, na linguagem Python.

```
letras == ["P", "F"]
for x = in letras
    {
            print (x)
    }
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado:

PF

98276239072 - FABIANO CARBONELL DA SILVA

Comentários:

Essa questão nos dá três chances de acertar. Vejamos...

Primeiro, a primeira linha apresenta um operador de comparação (==). A ideia não é comparar, mas – sim – atribuir valores à variável letras. Logo, o correto seria utilizar = em vez de ==;

Segundo, a segunda linha tem um sinal de igual desnecessário e não tem os dois-pontos ao final da declaração for – lembrando que eles são obrigatórios;

Terceiro, a terceira linha apresenta chaves para delimitar o bloco. Nós vimos insistentemente que a delimitação de blocos é feita por meio da indentação. Dessa forma, o código correto seria:

```
letras = ["P","F"]
for x in letras:
  print (x)
```

Gabarito: Errado

13. (CESPE / MPOG – 2013) Em Python, o comando int("1") cria um objeto do tipo int, que recebe 1 como parâmetro no seu construtor.

Comentários:

Essa é uma função de casting! *O que ela faz?* Bem, ela converte variáveis do tipo inteiro, ponto flutuante ou string em um inteiro. No caso, ele está recebendo um número como string (por conta das aspas) e convertendo em um inteiro. Esse número serve de parâmetro para o seu construtor (que é um método que cria um objeto, mas vocês não precisam saber disso).

Gabarito: Correto

14. (CESPE / ECT – 2012) A linguagem Python e seu interpretador estão disponíveis para as mais diversas plataformas. Para que seja usado em determinado sistema operacional não suportado, é possível gerar o Python a partir do programa fonte utilizando um compilador C. Nesse caso, o código fonte é traduzido para o formato bytecode, que é multiplataforma e pode ser distribuído de forma independente.

Comentários:

Ele realmente é multiplataforma. Além disso, permite que programas sejam compilados para um formato portável chamado de bytecode. Essa característica faz com que programas escritos nessa linguagem com uma biblioteca padrão sejam executadas da mesma forma em diversos sistemas operacionais que possuam um software interpretador de Python.

Gabarito: Correto

98276239072 - FABIANO CARBONELL DA SILVA

15. (CESPE / SERPRO – 2008) Python é uma linguagem livre de alto nível, orientada a objetos e de difícil leitura, pois não permite identação de linhas de código.

Comentários:

Ela realmente é de alto nível, orientada a objetos e de fácil leitura. Além disso, permite indentação de linhas de código.

Gabarito: Errado

98276239072 - FABIANO CARBONELL DA SILVA

Questionário de Revisão e Aperfeiçoamento

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma autoexplicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível! Vamos ao nosso questionário:

Perguntas

1. **O que é um algoritmo?**
2. **O que é uma linguagem de programação?**
3. **O que é um pseudocódigo?**
4. **O que é um fluxograma?**
5. **O que são variáveis em programação?**
6. **O que é uma constante?**
7. **Qual a função da estrutura de controle "if"?**
8. **Para que serve a estrutura de controle "else"?**
9. **O que é uma função em programação?**

98276239072 - FABIANO CARBONELL DA SILVA

10. O que é uma recursão?
11. O que é um loop "for"?
12. O que é um loop "while"?
13. O que é uma string?
14. O que é um array?
15. Qual a diferença entre uma lista e um array?
16. O que é um dicionário em Python?
17. Para que serve a função "print()" em Python?
18. O que é casting em programação?
19. O que são operadores aritméticos?
20. O que são operadores lógicos?
21. O que são operadores relacionais?
22. O que é a indentação em Python?
23. O que significa "case-sensitive"?
24. O que são parâmetros em uma função?
25. O que é um retorno de função?
26. O que é uma exceção em programação?
27. Como funciona o bloco "try-except" em Python?
28. O que são listas em Python?
29. O que é um conjunto (set) em Python?
30. O que é uma tupla em Python?
31. Qual a função da palavra-chave "def" em Python?
32. O que é uma linguagem de alto nível?
33. O que é uma linguagem de baixo nível?

98276239072 - FABIANO CARBONELL DA SILVA

34. O que é a linguagem de máquina?
35. O que é a Linguagem de Montagem (Assembly)?
36. O que é uma variável global?
37. O que é uma variável local?
38. O que é o comando "return"?
39. O que é um bloco "finally" em Python?
40. O que é um número ponto flutuante?
41. O que é um número inteiro?
42. O que é uma string literal?
43. O que são índices em Python?
44. O que são tipos booleanos?
45. Qual a diferença entre "==" e "=" em Python?
46. O que é uma lista aninhada em Python?
47. O que é uma chave em um dicionário?
48. O que significa iterar sobre uma lista?
49. O que é uma linguagem interpretada?
50. O que é um interpretador?
51. O que são comentários em código?
52. O que é uma função anônima (lambda)?
53. O que é o comando "break"?
54. O que é o comando "continue"?
55. O que são operadores de incremento e decremento?
56. O que é um IDE?
57. O que é a função input() em Python?

98276239072 - FABIANO CARBONELL DA SILVA

58. O que são funções built-in?

59. O que é uma string formatada em Python?

60. O que é concatenação de strings?

98276239072 - FABIANO CARBONELL DA SILVA

Perguntas com Respostas

1. O que é um algoritmo?

Um algoritmo é um conjunto de passos finitos e bem definidos para resolver um problema específico.

2. O que é uma linguagem de programação?

É uma forma de comunicação com o computador, usando comandos compreensíveis por ele.

3. O que é um pseudocódigo?

É uma forma simplificada de representar um algoritmo, sem usar a sintaxe rígida das linguagens de programação.

4. O que é um fluxograma?

É um diagrama usado para representar visualmente algoritmos ou processos, facilitando a compreensão.

5. O que são variáveis em programação?

São espaços na memória para armazenar valores, que podem ser alterados durante a execução de um programa.

6. O que é uma constante?

Uma constante é um valor fixo que não pode ser alterado durante a execução do programa.

7. Qual a função da estrutura de controle "if"?

Permite executar um bloco de código apenas se uma determinada condição for verdadeira.

8. Para que serve a estrutura de controle "else"?

O "else" é utilizado para definir o que deve ser feito caso a condição de um "if" não seja atendida.

9. O que é uma função em programação?

Uma função é um bloco de código reutilizável que executa uma tarefa específica quando invocado.

10. O que é uma recursão?

Recursão ocorre quando uma função chama a si mesma para resolver subproblemas de um problema maior.

98276239072 - FABIANO CARBONELL DA SILVA

11. O que é um loop "for"?

É uma estrutura de repetição que executa um bloco de código para um número específico de iterações.

12. O que é um loop "while"?

É uma estrutura de repetição que continua a execução enquanto uma condição for verdadeira.

13. O que é uma string?

Uma string é uma sequência de caracteres usados para representar textos em um programa.

14. O que é um array?

É uma estrutura de dados que armazena múltiplos valores do mesmo tipo em posições indexadas.

15. Qual a diferença entre uma lista e um array?

Uma lista pode conter diferentes tipos de dados, enquanto um array armazena apenas um tipo de dado.

16. O que é um dicionário em Python?

É uma estrutura de dados que associa chaves a valores, permitindo o acesso rápido a elementos por meio de suas chaves.

17. Para que serve a função "print()" em Python?

A função "print()" exibe na tela o valor de variáveis ou resultados de expressões.

18. O que é casting em programação?

Casting é a conversão de um valor de um tipo de dado para outro, como de string para inteiro.

19. O que são operadores aritméticos?

São símbolos que realizam operações matemáticas, como soma (+), subtração (-), multiplicação (*) e divisão (/).

20. O que são operadores lógicos?

São operadores que retornam valores booleanos (verdadeiro ou falso), como AND, OR e NOT.

21. O que são operadores relacionais?

98276239072 - FABIANO CARBONELL DA SILVA

Comparadores como >, <, >=, <=, ==, que avaliam relações entre dois valores e retornam verdadeiro ou falso.

22. O que é a indentação em Python?

Indentação é o recuo de linhas de código para definir blocos de código, essencial para a sintaxe do Python.

23. O que significa "case-sensitive"?

Significa que a linguagem diferencia letras maiúsculas de minúsculas, como no caso de variáveis.

24. O que são parâmetros em uma função?

Parâmetros são valores que são passados para uma função, permitindo que ela execute com diferentes entradas.

25. O que é um retorno de função?

O retorno de função é o valor que uma função envia de volta ao ponto onde foi chamada.

26. O que é uma exceção em programação?

É um erro ou evento inesperado que interrompe o fluxo normal de execução de um programa.

27. Como funciona o bloco "try-except" em Python?

É usado para capturar e tratar exceções, prevenindo que o programa pare devido a erros.

28. O que são listas em Python?

São coleções ordenadas de itens que podem ser alteradas (mutáveis) e permitem valores duplicados.

29. O que é um conjunto (set) em Python?

É uma coleção desordenada de itens únicos que não permite valores duplicados.

30. O que é uma tupla em Python?

É uma coleção ordenada de itens que não podem ser alterados (imutáveis) após sua criação.

31. Qual a função da palavra-chave "def" em Python?

É usada para definir uma nova função em Python.

32. O que é uma linguagem de alto nível?

98276239072 - FABIANO CARBONELL DA SILVA

É uma linguagem que se aproxima da linguagem humana, facilitando a programação, como o Python.

33. O que é uma linguagem de baixo nível?

É uma linguagem mais próxima do código de máquina, como Assembly, difícil de ser compreendida por humanos.

34. O que é a linguagem de máquina?

É o nível mais básico de linguagem que o computador entende diretamente, composta por 0s e 1s.

35. O que é a Linguagem de Montagem (Assembly)?

É uma linguagem de baixo nível que usa símbolos para representar comandos executados pelo computador.

36. O que é uma variável global?

É uma variável que pode ser acessada de qualquer parte do código, independentemente da função.

37. O que é uma variável local?

É uma variável declarada dentro de uma função e só pode ser acessada dentro dessa função.

38. O que é o comando "return"?

É usado para devolver um valor de uma função ao ponto onde ela foi chamada.

39. O que é um bloco "finally" em Python?

É um bloco opcional que é sempre executado no final de um bloco "try", independentemente de exceções.

40. O que é um número ponto flutuante?

É um número que possui uma parte decimal, como 3.14 ou -0.001.

41. O que é um número inteiro?

É um número sem parte decimal, podendo ser positivo ou negativo, como 10 ou -5.

42. O que é uma string literal?

É uma sequência de caracteres entre aspas que representa um texto, como "Olá Mundo!".

98276239072 - FABIANO CARBONELL DA SILVA

43. O que são índices em Python?

São números que identificam a posição de um item em uma lista ou string, começando por 0.

44. O que são tipos booleanos?

Tipos que só podem ter dois valores: True (verdadeiro) ou False (falso).

45. Qual a diferença entre "==" e "=" em Python?

"==" é um operador de comparação, enquanto "=" é um operador de atribuição.

46. O que é uma lista aninhada em Python?

É uma lista que contém outras listas como elementos, permitindo estruturas de dados mais complexas.

47. O que é uma chave em um dicionário?

É o identificador único que associa a um valor dentro de um dicionário.

48. O que significa iterar sobre uma lista?

É percorrer cada elemento da lista, geralmente usando loops como "for" ou "while".

49. O que é uma linguagem interpretada?

É uma linguagem cujo código é executado linha a linha, em vez de ser compilado como um todo.

50. O que é um interpretador?

É um software que executa um código de uma linguagem interpretada, como Python.

51. O que são comentários em código?

São anotações no código que explicam o funcionamento ou dão informações, ignoradas pelo interpretador.

52. O que é uma função anônima (lambda)?

É uma função sem nome que pode ser definida em uma única linha e usada em expressões simples.

53. O que é o comando "break"?

É usado para interromper a execução de um loop, saindo imediatamente do bloco.

54. O que é o comando "continue"?

98276239072 - FABIANO CARBONELL DA SILVA

É usado para pular a execução do restante do bloco e iniciar a próxima iteração de um loop.

55. O que são operadores de incremento e decremento?

São operadores que aumentam ou diminuem o valor de uma variável, como += e -=.

56. O que é um IDE?

IDE significa Integrated Development Environment, um ambiente que oferece ferramentas para facilitar a programação.

57. O que é a função input() em Python?

É uma função que captura a entrada do usuário durante a execução de um programa.

58. O que são funções built-in?

São funções nativas da linguagem de programação, como print(), len(), e type() no Python.

59. O que é uma string formatada em Python?

É uma string que permite a inserção de valores de variáveis diretamente no texto, utilizando "{}" ou f-strings.

60. O que é concatenação de strings?

É a união de duas ou mais strings em uma única, utilizando o operador "+" para juntar os textos.

98276239072 - FABIANO CARBONELL DA SILVA

LISTA DE QUESTÕES ESTRATÉGICAS

```
a = “Cagepa”
x = len(a)
x// = 3
print(x)
```

1. (CESPE / CAGEPA-PB – 2024) Assinale a opção que corresponde ao resultado da execução do código precedente, desenvolvido em Python.

a) 0,75
b) 3
c) 2
d) 0
e) Cag

2. (CESPE / SEPLAG-CE – 2024) Considerando linguagens de programação, julgue o item que se segue. Considere o código Python a seguir.

```
x = ("laranja","lima","taiti")
y = ["laranja","lima","taiti"]
print (type(x))
print (type(y))
```

Esse código, após executado, apresentará o seguinte resultado.

```
<class 'list'>
<class 'array'>
```

3. (CESPE / ANA – 2024) A linguagem Python é considerada dinamicamente tipada, pois os tipos das variáveis de seus programas podem ser modificados durante a execução.

4. (CESPE / TRT8 – 2022) Na linguagem de programação Python, as funções:

a) retornam somente objetos.
b) não aceitam parâmetros opcionais (com *defaults*).
c) não podem ter suas propriedades alteradas.
d) não aceitam *doc strings*.
e) aceitam que os parâmetros sejam passados com nome, não importando a ordem em que os parâmetros foram passados.

5. (CESPE / PC-PB - 2022) Python é uma linguagem procedural que utiliza quatro tipos de dados predefinidos para lidar com coleções: conjuntos, dicionários, listas e tuplas. A respeito desses tipos de dados, julgue os itens a seguir.

I O conjunto permite o armazenamento de uma tupla, mas não o de uma lista.

98276239072 - FABIANO CARBONELL DA SILVA

II A tupla é idêntica à lista, exceto pela forma mais simples com que sua declaração é realizada.
III A lista é um tipo de dados variável que permite a alteração de seus elementos após a sua criação.

Assinale a opção correta.

a) Apenas o item I está certo.
b) Todos os itens estão certos.
c) Apenas o item II está certo.
d) Apenas os itens I e III estão certos.
e) Apenas os itens II e III estão certos.

6. **(CESPE / DPE-RO – 2021) Na linguagem Python, são consideradas sequências mutáveis as:**

a) strings.
b) cadeias.
c) ranges.
d) tuplas.
e) listas.

7. **(CESPE / SERPRO – 2021) As tuplas, embora sejam semelhantes às listas, estão limitadas a, no máximo, cinco níveis.**

8. **(CESPE / SERPRO – 2021) Listas são coleções alteráveis de qualquer tipo de objeto – como, por exemplo, outras listas – capazes de gerar um efeito top-down sem limite de níveis.**

9. **(CESPE – SEED-PR / 2021) Em Python, quando mais de um operador aparece em uma expressão, a ordem de avaliação depende das regras de precedência de cada linguagem. Assim, ao programar em Python, além de observar essas regras, é preciso considerar, ainda, a forma como a linguagem representa seus operadores, conforme demonstrado nos comandos a seguir.**

x=7*32%4 print (x)**

Assinale a opção que corresponde à saída que o compilador Python apresentará para os comandos em questão:

a) 1
b) 3
c) 7
d) 15
e) 15.75

10. **(CESPE – SEED-PR / 2021) Considere o programa a seguir, na linguagem Python.**

98276239072 - FABIANO CARBONELL DA SILVA

```
if 5 > 2
{
print("True!")
}
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado.

True!

11. (CESPE / Polícia Federal – 2021) O código Python a seguir apresenta como resultado "TRUE".

```
x = bool(-3)
y = bool("True"*x)
z = bool("False")
print(x and y and z)
```

12. (CESPE – Polícia Federal / 2018) Considere o programa a seguir, na linguagem Python.

```
letras == ["P", "F"]
for x = in letras
    {
            print (x)
    }
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado:

PF

13. (CESPE / MPOG – 2013) Em Python, o comando int("1") cria um objeto do tipo int, que recebe 1 como parâmetro no seu construtor.

14. (CESPE / ECT – 2012) A linguagem Python e seu interpretador estão disponíveis para as mais diversas plataformas. Para que seja usado em determinado sistema operacional não suportado, é possível gerar o Python a partir do programa fonte utilizando um compilador C. Nesse caso, o código fonte é traduzido para o formato bytecode, que é multiplataforma e pode ser distribuído de forma independente.

15. (CESPE / SERPRO – 2008) Python é uma linguagem livre de alto nível, orientada a objetos e de difícil leitura, pois não permite identação de linhas de código.

98276239072 - FABIANO CARBONELL DA SILVA

GABARITO

1. LETRA C
2. ERRADO
3. CORRETO
4. LETRA E
5. LETRA D
6. LETRA E
7. ERRADO
8. CORRETO
9. LETRA B
10. ERRADO
11. ERRADO
12. ERRADO
13. CORRETO
14. CORRETO
15. ERRADO

98276239072 - FABIANO CARBONELL DA SILVA

Referências Bibliográficas

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: https://docs.python.org/3/. Acesso em: 24 set. 2024.

98276239072 - FABIANO CARBONELL DA SILVA

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.

Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.